PROJECT 03

AI Smart Device Control & Automation Assistant

STEP 02 PROJECT DOCUMENTATION — Configuration & Device Command Model

Item	Current Project Details
Project	AI Smart Device Control & Automation Assistant
Completed Steps	Step 01 — Initialization; Step 02 — Configuration & Command Model
Primary Language	Python
Environment	Python virtual environment (venv)
Configuration	config/config.json
Command Model	src/command_model.py
Configuration Loader	src/config_loader.py
Step 02 Status	**COMPLETED — PASS**

1. Executive Summary

Step 02 established the configuration layer and structured device-command model for Project 03. A JSON configuration file was created to define project settings, enabled device categories, assistant behavior, and logging. A reusable **DeviceCommand** data model was then implemented to represent device, action, target, and optional value fields. Both the command-model validation and configuration-loader validation were executed successfully.

2. Configuration Architecture

```
config/config.json ↓ Configuration Loader ↓ Project + Device + Assistant Settings User Command ↓
DeviceCommand ↓ device / action / target / value
```

The configuration and command layers are intentionally separated from future AI and hardware-control modules. This allows later voice or AI processing to produce structured commands without directly coupling natural-language input to device execution.

3. Configuration Model

The Step 02 configuration defines the development environment and the initial device categories that the future assistant can control. The three planned device groups are laptop, Bluetooth, and IoT.

Configuration Area	Value
Project Name	AI Smart Device Control & Automation Assistant
Version	0.1.0
Environment	development
Laptop	enabled: true
Bluetooth	enabled: true
IoT	enabled: true
Response Mode	text
Logging	enabled: true

4. Device Command Model

The **DeviceCommand** model provides a common structure for commands that will later be generated by manual input, AI intent analysis, or voice processing. The implemented fields are:

Field	Purpose
device	Identifies the device category, such as iot.
action	Defines the requested operation, such as turn_on.
target	Identifies the target device or resource, such as desk_light.
value	Optional value for commands that require additional data.

5. Example Structured Command

```
{ "device": "iot", "action": "turn_on", "target": "desk_light", "value": null }
```

This example demonstrates the intended separation between natural-language understanding and device execution. A future AI layer can convert a request such as “Turn on the desk light” into this structured representation.

6. Configuration Loader

The configuration loader reads the JSON configuration using a project-root-relative path. It verifies that the configuration file exists and exposes the loaded project, device, and assistant settings to the application.

```
python .\src\config_loader.py
```

7. Step 02 Validation — Command Model

The command-model test was executed successfully. The program instantiated a DeviceCommand object and converted the structured data to formatted JSON.

Test	Observed Result	Status
DeviceCommand creation	Command object created successfully	**PASS**
Dictionary conversion	Structured command generated	**PASS**
JSON conversion	Readable JSON output generated	**PASS**
Step verification	Step 02 validation completed	**PASS**

8. Step 02 Validation — Configuration Loader

The configuration-loader test was executed successfully. The program loaded config/config.json and displayed the project name, version, enabled device categories, response mode, and logging setting.

Test	Observed Result	Status
Configuration file lookup	config/config.json found	**PASS**
JSON loading	Configuration parsed successfully	**PASS**
Project settings	Name and version displayed	**PASS**
Device settings	Laptop, Bluetooth, IoT displayed as enabled	**PASS**
Assistant settings	Text response and logging displayed	**PASS**

9. Development Commands

```
# Command model python .\src\command_model.py # Configuration loader python .\src\config_loader.py
```

Both Step 02 verification commands completed successfully. No AI API, Bluetooth control package, or IoT communication dependency was added at this stage.

10. Project Status After Step 02

Milestone	Status
Project initialization	**COMPLETED**
Virtual environment	**COMPLETED**
Configuration file	**COMPLETED**
Configuration loader	**COMPLETED**
Device command model	**COMPLETED**
Command JSON serialization	**PASS**
Configuration validation	**PASS**
AI intent processing	Pending
Laptop automation	Pending
Bluetooth control	Pending
IoT control	Pending

11. Troubleshooting / Lessons Learned

Area	Step 02 Observation
Configuration separation	Settings are kept outside Python source code in JSON format.
Command abstraction	Device commands are represented as structured data before execution.
Future AI integration	AI can later generate a DeviceCommand rather than directly calling hardware functions.
Dependency control	No additional feature dependency was introduced before it was required.

12. Security Baseline

Step 02 contains no API credentials or private secrets. The configuration file contains only non-sensitive project settings. Future credentials will be kept outside source code and public documentation.

13. Step 02 Conclusion

STEP 02 — CONFIGURATION & COMMAND MODEL: PASS. The project now has a reusable configuration layer and a structured device-command representation. These components provide the foundation for the next stage, where command parsing and intent interpretation can be introduced.

14. Revision Record

Revision	Update
STEP 01	Project initialization, environment setup, validation, roadmap, and documentation foundation.
STEP 02	Added config/config.json, configuration loader, DeviceCommand model, validation results, updated milestone status, and Step 02 conclusion.
Next	Update this same report after Step 03 is completed.